

Guia Completo: Sistema de Análise de Futebol com IA/ML

Este documento foi elaborado para fornecer todas as informações, etapas e conceitos necessários para construir um sistema completo de análise de partidas de futebol usando Inteligência Artificial e Visão Computacional. Baseado no tutorial "Build an AI/ML Football Analysis system with YOLO, OpenCV, and Python", este guia é ideal para ser utilizado como material de estudo e implementação em um processo seletivo.

1. Visão Geral do Projeto

O objetivo do projeto é construir um pipeline de Visão Computacional capaz de analisar vídeos de partidas de futebol (visão tática/aérea) e extrair estatísticas avançadas. O sistema realiza as seguintes tarefas:

- Detecta e rastreia jogadores, árbitros e a bola através de todo o vídeo.
- Segmenta os jogadores e os atribui às suas respectivas equipes baseando-se na cor da camisa.
- Mede a porcentagem de posse de bola de cada equipe.
- Calcula e compensa o movimento da câmera (Optical Flow) para garantir medições precisas.
- Realiza uma transformação de perspectiva (Perspective Transformation) para converter pixels distorcidos da câmera em distâncias reais (metros).
- Mede a velocidade (km/h) e a distância total percorrida (metros) por cada jogador.

2. Configuração do Ambiente e Requisitos

Para reproduzir este projeto, você precisará configurar um ambiente Python com bibliotecas específicas de Machine Learning e Visão Computacional.

Bibliotecas Essenciais

Biblioteca	Uso no Projeto
ultralytics	Implementação e inferência dos modelos YOLOv8 e YOLOv5 para detecção de objetos.
supervision	Utilizada para o rastreamento de objetos (ByteTrack) e formatação conveniente das detecções.
opencv-python (cv2)	Processamento de vídeo, desenho de anotações, optical flow e transformação geométrica.
scikit-learn	Utilizada pelo algoritmo K-Means para clusterização das cores das camisas dos jogadores.
pandas	Interpolação de posições ausentes da bola (preenchimento de falhas de detecção).
numpy	Manipulação de arrays, coordenadas matemáticas e matrizes de imagem.
roboflow	Download e gerenciamento do dataset customizado para fine-tuning do modelo YOLO.

Dados e Modelos Necessários

- **Vídeo de Entrada:** Dataset "DFL Bundesliga Data Shootout" (encontrado no Kaggle). Utilize clipes com visão tática (Eagle Eye).
- **Dataset de Treinamento:** "Football Player Detection Image Dataset" (disponível no Roboflow). Contém anotações específicas para separar jogadores, árbitros, goleiros e a bola.

- **Modelo Pré-treinado:** YOLOv5 (versão Extra Large - yolov5x) é recomendado no tutorial para melhor performance de precisão após o fine-tuning.

3. Pipeline do Projeto (Passo a Passo)

Passo 1: Detecção de Objetos (YOLO)

Modelos YOLO pré-treinados genéricos costumam perder a bola em muitos frames e detectam pessoas fora de campo. Para resolver isso, realizamos o *fine-tuning* do YOLO.

1. Baixe o dataset específico via Roboflow.
2. Treine o modelo (recomendado usar Google Colab devido à necessidade de GPU). O tutorial realiza 100 épocas de treinamento usando YOLOv5x.
3. Obtenha o arquivo de pesos best.pt e utilize-o para realizar a inferência no vídeo. As classes de saída serão: Bola (0), Goleiro (1), Jogador (2) e Árbitro (3).

Passo 2: Rastreamento de Objetos (ByteTrack)

Detectar frame a frame não é suficiente; é preciso saber que um jogador no frame 1 é o mesmo no frame 2. Isso se chama Rastreamento (Tracking).

- Utilizamos o ByteTrack da biblioteca supervision.
- Antes do rastreamento, o ID da classe "Goleiro" (1) é sobrescrito e tratado como "Jogador" (2) para facilitar a atribuição de times, pois estatísticas separadas para goleiros não serão o foco.
- O rastreador gera um track_id único que se mantém consistente para o objeto em movimento.

Passo 3: Anotações Personalizadas

Bounding boxes convencionais poluem visualmente o vídeo de futebol. A abordagem adotada é desenhar formas geométricas usando OpenCV (cv2.ellipse, cv2.drawContours):

- **Jogadores:** Uma elipse (círculo achatado) na base do jogador (nos pés).
- **Árbitros:** Elipse amarela na base.
- **Bola:** Um triângulo invertido apontando para a bola, com borda.
- **Track ID:** Um pequeno retângulo preenchido contendo o ID do jogador (track_id).

Passo 4: Atribuição de Equipes (K-Means Clustering)

É necessário separar os jogadores do time A e do time B dinamicamente:

1. Recorta-se (crop) a imagem do jogador usando a bounding box.
2. Pega-se apenas a **metade superior** do crop, pois é lá que está a camisa (evitando shorts e meias).
3. Roda-se um **K-Means (k=2)** no array de pixels para separar a cor da camisa da cor do gramado (background). O cluster que domina os cantos do crop é considerado gramado; o outro é a camisa.
4. Roda-se um **segundo K-Means (k=2)** sobre todas as cores de camisas encontradas no primeiro frame para descobrir as duas cores principais dos times.
5. Cada jogador recebe a cor e a identificação do time, colorindo a elipse correspondente aos pés.

Passo 5: Posse de Bola e Interpolação

A bola, por ser pequena e rápida, sofre de detecções perdidas ou borradas (motion blur). Para lidar com isso:

- **Interpolação:** Os dados da bola (coordenadas x,y) são convertidos num DataFrame Pandas e o método `.interpolate()` é usado para preencher os valores ausentes assumindo uma trajetória linear.
- **Atribuição:** Calcula-se a distância euclidiana (em pixels) entre o centro da bola e os pés de cada jogador. Se a bola estiver abaixo de um limite (ex: 70 pixels), o jogador é considerado "com a posse de bola".
- **Estatística de Tempo:** Acumula-se os frames de posse por time para desenhar um painel semitransparente na tela mostrando o percentual (%) de posse.

Passo 6: Compensação do Movimento da Câmera (Optical Flow)

Ao analisar velocidade, a câmera seguindo a bola (pan) distorce a posição do jogador. É crucial medir o quanto a câmera se moveu e subtrair esse valor das coordenadas dos jogadores.

1. Cria-se uma máscara isolando áreas estáticas da tela (faixas nas extremidades superior e inferior do vídeo), evitando focar nos jogadores.
2. Usa-se `cv2.goodFeaturesToTrack` para detectar "corners" (cantos/pontos notáveis) nessas áreas estáticas.
3. Calcula-se o movimento desses pontos de um frame para outro com `cv2.calcOpticalFlowPyrLK` (Algoritmo Lucas-Kanade).

4. A maior distância percorrida por essas features indica o deslocamento [dx, dy] da câmera, que é então usado para ajustar (subtrair) a posição do jogador.

Passo 7: Transformação de Perspectiva (Metrificação Real)

A imagem do vídeo é bidimensional e tem profundidade (perspectiva). Um pixel no topo da tela não equivale ao mesmo número de metros físicos que um pixel na base. Para corrigir isso:

- Mapeiam-se os cantos de uma área trapezoidal conhecida (ex: uma porção do gramado baseada nas linhas visíveis).
- Converte-se esse polígono da câmera para um retângulo plano de dimensões reais, sabendo que o campo mede aproximadamente 105m x 68m (um quadrante medirá proporções exatas, ex: 23.3m x 68m).
- Usa-se `cv2.getPerspectiveTransform` para gerar uma matriz de transformação.
- Com `cv2.perspectiveTransform`, todas as coordenadas reajustadas dos pés dos jogadores são convertidas para **metros no mundo real**.

Passo 8: Cálculo de Velocidade e Distância

Com as posições transformadas em metros reais:

- Agrupa-se a variação de posição a cada **5 frames** (frame window) para atenuar ruídos nas detecções.
- **Distância Coberta (m)**: A distância euclidiana entre a posição (x,y) real do jogador no frame N e frame N+5.
- **Tempo (s)**: Calcula-se baseado na taxa de quadros (FPS). (Ex: 5 frames / 24 fps = ~0.2s).
- **Velocidade**: (Distância / Tempo) * 3.6 (para converter de m/s para km/h).
- O script acumula as distâncias para encontrar a "Distância Total Percorrida" e plota ambos os valores (Velocidade e Distância) abaixo das caixas/elipses dos jogadores na tela final.

4. Estrutura de Código Sugerida (Módulos)

A organização de um projeto de Machine Learning é crucial para processos seletivos. Utilize Programação Orientada a Objetos e separe a lógica nos seguintes módulos (pastas e arquivos):

```
football_analysis_project/
```

```

|
|— main.py          # Orquestrador do pipeline (lê o vídeo, invoca módulos, exporta)
|— yolov5/          # (Opcional) Submódulo com clone do modelo YOLOv5
|— models/          # Pasta para armazenar best.pt e last.pt do YOLO treinado
|— input_videos/    # Onde os vídeos em .mp4 cru serão armazenados
|— output_videos/   # Onde o sistema salvará o .avi com os desenhos sobrepostos
|
|— utils/           # Módulo de utilitários
|  |— video_utils.py # Funções read_video e save_video
|  |— bbox_utils.py  # Medições (distância, largura, centro de bounding boxes)
|
|— trackers/        # Módulo de detecção e rastreamento
|  |— tracker.py     # Classe principal (integra Ultralytics + Supervision + Desenhos)
|
|— team_assigner/   # Módulo de clusterização de K-Means
|  |— team_assigner.py # Crop, segmentação e K-Means da camisa
|
|— player_ball_assigner/ # Módulo lógico de jogo
|  |— player_ball_assigner.py # Liga a bola interpolada ao jogador mais próximo
|
|— camera_movement_estimator/ # Módulo Optical Flow
|  |— camera_movement_estimator.py
|
|— view_transformer/ # Módulo de Perspectiva (Geometria do mundo real)
|  |— view_transformer.py
|
|— speed_and_distance_estimator/ # Módulo final de Estatísticas
|  |— speed_and_distance_estimator.py

```

5. Dicas para o Processo Seletivo

- **Otimize o Fluxo de Trabalho (Stubs/Pickle):** Operações de IA e Optical Flow são demoradas. O tutorial usa arquivos .pickle para salvar as matrizes de rastreamento no disco após a primeira execução. Demonstre isso no seu código para mostrar que se importa com tempo de iteração e performance no desenvolvimento.
- **Tratamento de Exceções Lógicas:** Preste atenção em condições fora da quadra

geométrica. O sistema retorna None se o jogador estiver fora do polígono de perspectiva real para evitar erros matemáticos ao calcular velocidade. Mostre preocupação com esses *Edge Cases*.

- **Explicabilidade e Orientação a Objetos:** Um dos pontos fortes deste tutorial é que cada comportamento complexo ganha uma Classe e um Módulo próprios. Comentar sobre Inversão de Dependências (SOLID) ou acoplamento e usar o main.py estritamente como orquestrador contará muitos pontos em avaliações de código (Code Review).
- **Não Tente Adivinhar Hiperparâmetros:** A configuração de K-Means e Optical Flow requer tentativas. Documente (usando comentários ou um README.md) por que você escolheu parâmetros como maxLevel=2 no fluxo óptico ou as tolerâncias de distância máxima para posse de bola (ex: 70 pixels). Mostrar o racional de sua escolha vale tanto quanto a própria solução técnica.